



Experiment 9

Student Name: Tejinderpal Singh

Branch: CSE

Semester: 6th

Subject Name: System Design

UID: 23BCS80343

Section/Group: KRG 3-B

Date of Performance: 23/04/2026

Subject Code: 23CSH-314

1. Aim: To design and analyze a scalable **Movie Recommendation System (like Netflix/Amazon Prime)** that suggests personalized movies to users based on preferences, watch history, and ratings using distributed system architecture.

2. Objective:

- To identify functional and non-functional requirements of a recommendation system
- To design High Level Architecture (HLD) for movie platforms
- To understand recommendation algorithms (content-based & collaborative filtering)
- To design microservices for user, movie, and recommendation management
- To study real-time user activity tracking
- To understand scalability challenges in large-scale systems

3. Tools:

Draw.io / Excalidraw - System design diagrams

MySQL / PostgreSQL - User & movie data

Redis - Caching

Kafka - Event streaming (user activity)

Load Balancer - Traffic distribution

Microservices Architecture

ElasticSearch - Fast searching

ML Models - Recommendation engine

4. System Requirements:

A. Functional Requirements:-

- Users should be able to register and login
- Users should view movie list and details
- Users should search movies
- Users should rate and review movies
- System should recommend personalized movies
- Users should create watchlist
- System should track watch history
- System should show trending/popular movies



B. Non-Functional Requirements:-

1) Scalability

- a. Support millions of users
- b. Handle large movie database

2) Consistency (CAP Theorem)

- a. Eventual consistency acceptable
- b. Accurate recommendations

3) Availability

- a. Recommendations should always be available

4) Latency

- a. Recommendation response < 200 ms
- b. Search response < 100 ms

5) Reliability

- a. No loss of user data
- b. System should handle failures

5. High Level Design (HLD):

System follows Microservices Architecture

Main Components:

1) API Gateway

- a. Authentication
- b. Authorization
- c. Routing
- d. Rate Limiting

2) User Service

- a. Registration & login
- b. User profile management
- c. Stores user preferences
- d. User DB (MySQL)

3) Movie Service

- a. Stores movie data
- b. Fetch movie details
- c. Movie DB

4) Recommendation Service

- a. Provides personalized suggestions
- b. Uses collaborative filtering
- c. Uses content-based filtering



- 5) Rating & Review Service
 - a. Stores ratings
 - b. Stores reviews
 - c. Updates recommendation system
- 6) Watchlist Service
 - a. Add/remove movies
 - b. Personalized list
- 7) Search Service
 - a. Search by name, genre, actor
 - b. Uses ElasticSearch
- 8) Activity Tracker (Kafka)
 - a. Tracks user behavior
 - b. Sends data to recommendation engine
- 9) Redis (Caching)
 - a. Cache recommendations
 - b. Cache popular movies
- 10) ML Model / Recommendation Engine
 - a. Processes user data
 - b. Generates recommendations
- 11) Database Layer
 - a. User DB
 - b. Movie DB
 - c. Interaction DB
- 12) Load Balancer
 - a. Distributes traffic
 - b. Ensures high availability

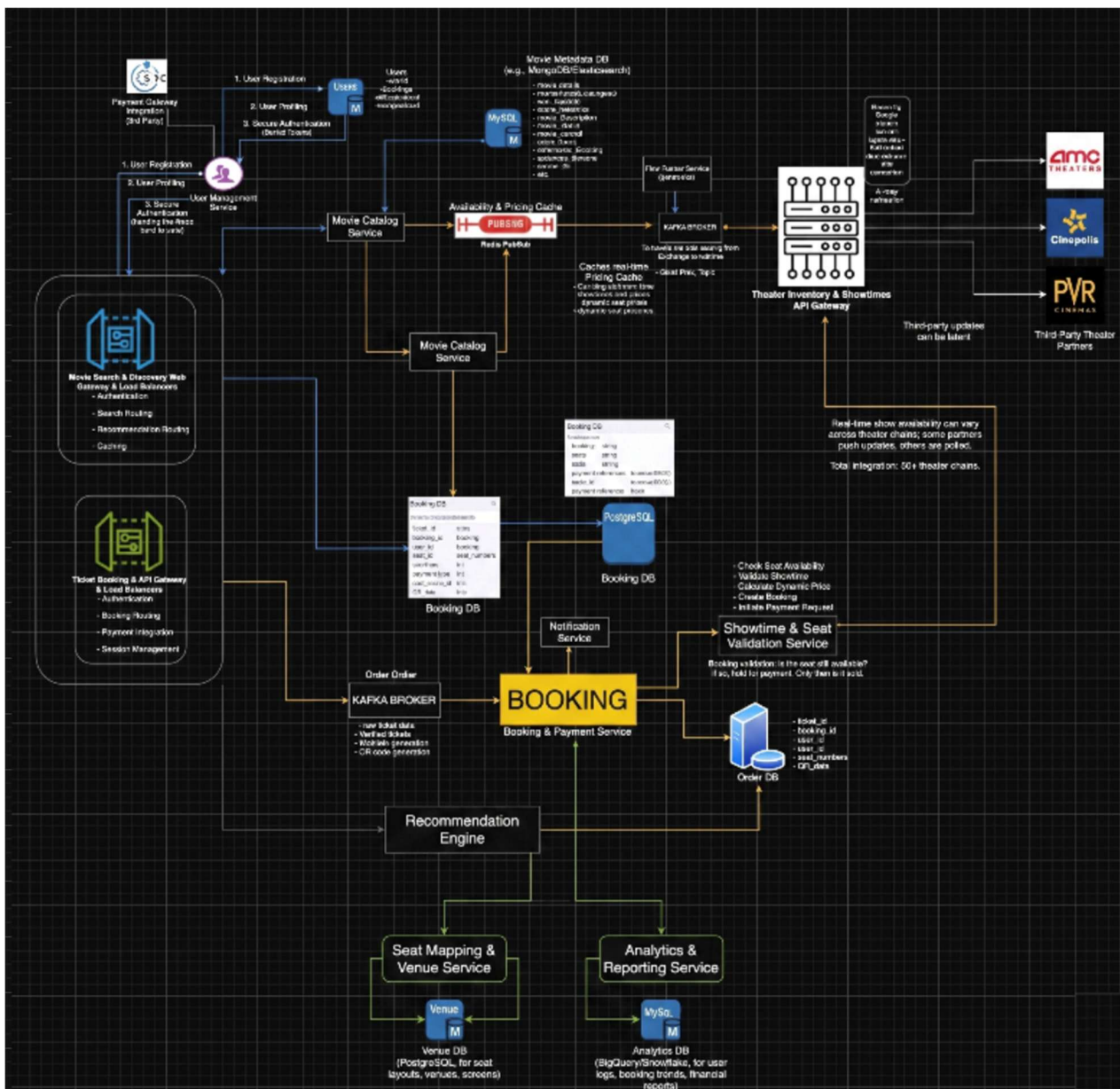
6. Recommendation Flow (Step-by-Step):

- User logs in
- User browses or searches movies
- User watches/rates a movie
- Activity sent to Kafka
- Data processed by recommendation engine
- Personalized recommendations generated
- User receives recommended movies

7. Scalability Solution:

To support millions of users:

- Use Kafka for user activity streaming
- Use Redis for caching recommendations
- Use Load Balancers for traffic distribution
- Use Microservices architecture
- Use separate databases (User, Movie, Interaction)
- Use ElasticSearch for fast search
- Use API Gateway for centralized control



8. Learning Outcomes (What I Have Learnt):

- Learned how recommendation systems are designed
- Understood content-based and collaborative filtering
- Learned real-time user tracking using Kafka
- Understood personalized recommendation logic
- Learned importance of scalability in large systems
- Understood microservices architecture
- Learned how platforms like Netflix work